Click here to view the PDF version.

```julia
begin
    using LinearAlgebra
    using PlutoUI
    using PlutoTeachingTools
    using Plots
    using LaTeXStrings
    using HypertextLiteral: @htl, @htl_str
end
```

## Table of Contents


**Iterative methods for linear systems**

Richardson iterations

Computational cost

**⚠ TODO ⚠**

Convergence analysis

Choosing a good preconditioner

**⚠ TODO ⚠**

Error control and stopping criterion

Jacobi and Gauss-Seidel method

Linear systems involving symmetric positive-definite matrices

Steepest descent method

**⚠ TODO ⚠**

Preconditioned steepest descent

**⚠ TODO ⚠**

Conjugate gradient method

**⚠ TODO ⚠**

**⚠ TODO ⚠**

# Iterative methods for linear systems

In the previous notebook we looked at direct methods for solving linear systems $\mathbf{A}\mathbf{x} = \mathbf{b}$ based on LU factorisation of the system matrix $\mathbf{A}$. We saw that even for sparse matrices we may need $O(n^3)$ computational time to perform the factorisation and $O(n^2)$ memory and to store the $\mathbf{L}$ and $\mathbf{U}$ factors due to fill in.

Both can make the **solution of linear systems prohibitively expensive for large matrices**. In this notebook we will develop *iterative methods*. These build a sequence of solution vectors $\mathbf{x}^{(k)}$, which is designed to converge to the solution $\mathbf{x}$, that is $\lim_{k\to\infty} \mathbf{x}^{(k)} = \mathbf{x}$. Typically these methods are based on performing matrix-vector products $\mathbf{A}\mathbf{v}$ in each iteration step. Exactly this difference to the direct methods — that is *not* employing the matrix $\mathbf{A}$, but only the matrix-vector product — is what often leads to an overall reduction of the computational cost.

## Richardson iterations

A general framework for building iterative methods for solving linear systems is Richardson's method. The idea is to introduce a **matrix splitting** $\mathbf{A} = \mathbf{P} - \mathbf{R}$ where $\mathbf{P}$ is an invertible matrix and $\mathbf{R} = \mathbf{P} - \mathbf{A}$. With this idea we rewrite $\mathbf{A}\mathbf{x} = \mathbf{b}$ to

$$(\mathbf{P} - \mathbf{R})\mathbf{x} = \mathbf{b} \qquad \Leftrightarrow \qquad \mathbf{P}\mathbf{x} = (\mathbf{P} - \mathbf{A})\mathbf{x} + \mathbf{b} \qquad \Leftrightarrow \qquad \mathbf{x} = \mathbf{P}^{-1}(\mathbf{P} - \mathbf{A})\mathbf{x} + \mathbf{b}$$

If we define $g(\mathbf{x}) = \mathbf{P}^{-1}(\mathbf{P} - \mathbf{A})\mathbf{x} + \mathbf{b}$ we observe this to be exactly the setting of the multi-dimensional fixed-point methods we developed in chapter 4, i.e. our goal is to obtain a solution $\mathbf{x}_*$ with $\mathbf{x}_* = g(\mathbf{x}_*) = \mathbf{P}^{-1}(\mathbf{P} - \mathbf{A})\mathbf{x}_* + \mathbf{b}$.

Starting from an initial vector $\mathbf{x}^{(0)}$ we thus iterate

$$\mathbf{P}\mathbf{x}^{(k+1)} = (\mathbf{P} - \mathbf{A})\mathbf{x}^{(k)} + \mathbf{b}, \qquad k = 0, 1, \ldots, \tag{2}$$

that is in each step we solve the linear system $\mathbf{P}\mathbf{x}^{(k+1)} = \widetilde{\mathbf{b}}^{(k)}$ where $\widetilde{\mathbf{b}}^{(k)} = (\mathbf{P} - \mathbf{A})\mathbf{x}^{(k)} + \mathbf{b}$. Assuming this sequence converges, i.e. that $\lim_{k\to\infty} \mathbf{x}^{(k)} = \mathbf{x}_*$, then

$$\mathbf{P}\mathbf{x}_* = (\mathbf{P} - \mathbf{A})\mathbf{x}_* + \mathbf{b} \qquad \Leftrightarrow \qquad \mathbf{0} = \mathbf{b} - \mathbf{A}\mathbf{x}_*,$$

i.e. the limit $\mathbf{x}_*$ is indeed the solution to the original equation.

The matrix $\mathbf{P}$ is usually referred to as the **preconditioner**. Since we need to solve a linear system $\mathbf{P}\mathbf{u}^{(k)} = \mathbf{r}^{(k)}$ in *each iteration* $k$, an important criterion for a good preconditioner is that solving such linear systems is cheap.

Let us rewrite equation (2) as

$$\mathbf{P}\underbrace{\left(\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\right)}_{=\mathbf{u}^{(k)}} = \underbrace{\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}}_{=\mathbf{r}^{(k)}}.$$

The quantity $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$ appearing in this expression is the **residual** at iteration $k$. As usual it measures how far the iterate $\mathbf{x}^{(k)}$ is from being a solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$: if $\mathbf{r}^{(k)} = \mathbf{0}$, then indeed $\mathbf{A}\mathbf{x}^{(k)} = \mathbf{b}$, i.e. that $\mathbf{x}^{(k)}$ is the solution.

We formulate Richardson's iterations:

**Algorithm 1: Richardson iteration**

Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a system matrix, right-hand side $\mathbf{b} \in \mathbb{R}^n$, an invertible preconditioner $\mathbf{P} \in \mathbb{R}^{n\times n}$ as well as the desired convergence tolerance $\varepsilon$. For $k = 0, 1, \ldots$ iterate:

1. Compute residual $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$
2. Solve linear system $\mathbf{P}\mathbf{u}^{(k)} = \mathbf{r}^{(k)}$ for the update $\mathbf{u}^{(k)}$.
3. Compute new $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{u}^{(k)}$ .

The iteration is stopped as soon as $\frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|} < \varepsilon$.

We will discuss the rationale behind the stopping condition $\|\mathbf{r}^{(k)}\|/\|\mathbf{b}\| < \varepsilon$ in the subsection on error control below.

Comparing with our discussion on the fixed point methods we studied in chapter 4 we notice that Algorithm 1 is essentially fixed-point iteration $\mathbf{x}^{(k+1)} = g(\mathbf{x}^{(k)})$ with the map $g$ given by

$$g(\mathbf{x}) = \mathbf{x} + \mathbf{P}^{-1}\underbrace{(\mathbf{b} - \mathbf{A}\mathbf{x})}_{=\mathbf{r}} \tag{3}$$

and as discussed above the fixed point $\mathbf{x}_* = g(\mathbf{x}_*)$ necessarily satisfies $\mathbf{0} = \mathbf{r}_* = \mathbf{b} - \mathbf{A}\mathbf{x}_*$ and therefore is a solution to the linear system.

An implementation of Algorithm 1 in Julia is given by

```
richardson (generic function with 1 method)
```

```julia
function richardson(A, b, P; x=zero(b), tol=1e-6, maxiter=100)
    history = Float64[]  # Track relative residual norm
    for k in 1:maxiter
        r = b - A * x

        relnorm = norm(r) / norm(b)
        push!(history, relnorm)
        if relnorm < tol
            break
        end

        u = P \ r
        x = x + u
    end
    (; x, history)  # Return current iterate and history
end
```

Let us consider the test problem $\mathbf{Ax} = \mathbf{b}$ with

```
A =
100×100 Matrix{Float64}:
 13.497     -0.0373554   0.372059    0.455755   …  -0.993431    1.72989   -0.500089
  0.339416  40.9764     -0.49252    -0.632024       0.0242995   1.32131    0.53068
  0.883111  -0.238479   39.8346      0.082532      -0.27617     0.522972  -1.09944
 -0.860942   1.01789    -0.405506   49.4443         1.36896    -1.25262    0.131916
 -0.910278   2.46261     0.0137423  -0.0335643     -0.0064238  -0.652895   0.485895
  1.99898   -1.65667    -0.0638512  -3.35575    …   0.390615    0.238168   0.693313
  0.930066   0.801534    1.35834     0.523876      -0.436435   -0.866964   0.239833
  ⋮                                                ⋱
  0.639166  -0.136747   -0.163707   -0.109682      -0.190203   -0.528371   0.946597
  0.270023  -0.186965   -0.888258    0.0374101  …  -1.30121     1.30031   -0.516153
 -1.24605   -0.348588   -1.03105    -0.276275       1.21323     0.394299  -1.35424
 -1.13901   -1.02876    -1.02613     0.158151      46.866       0.480092  -0.478084
 -0.553396  -0.264033    1.19852    -0.626974       0.739913   34.3639     1.75868
  1.05441    0.760557   -0.669326    0.381245      -1.04612    -1.48061   27.5318
```

```julia
# Generate a random matrix, which has large entries on the diagonal
A = randn(100, 100) + Diagonal(10 .+ 50rand(100))
```

```julia
b = rand(100);
```

In this case the problem is sufficiently small that it's reference can still be computed by a direct method, e.g. the LU factorisation performed by default when employing Julia's `\` operator:

```
x_reference =
▸[0.0627659, 0.00185212, 0.0268642, 0.00660236, -0.00137215, 0.00123146, 0.00509246, 0.0
```

```julia
x_reference = A \ b
```

To apply Richeardson iterations, we need a preconditioner $\mathbf{P}$. Due to the construction of $\mathbf{A}$ we chose it has its largest entries in each row on the diagonal. A particularly simple preconditioner, which typically works well for such matrices is to use the diagonal of $\mathbf{A}$ as $\mathbf{P}$, i.e.

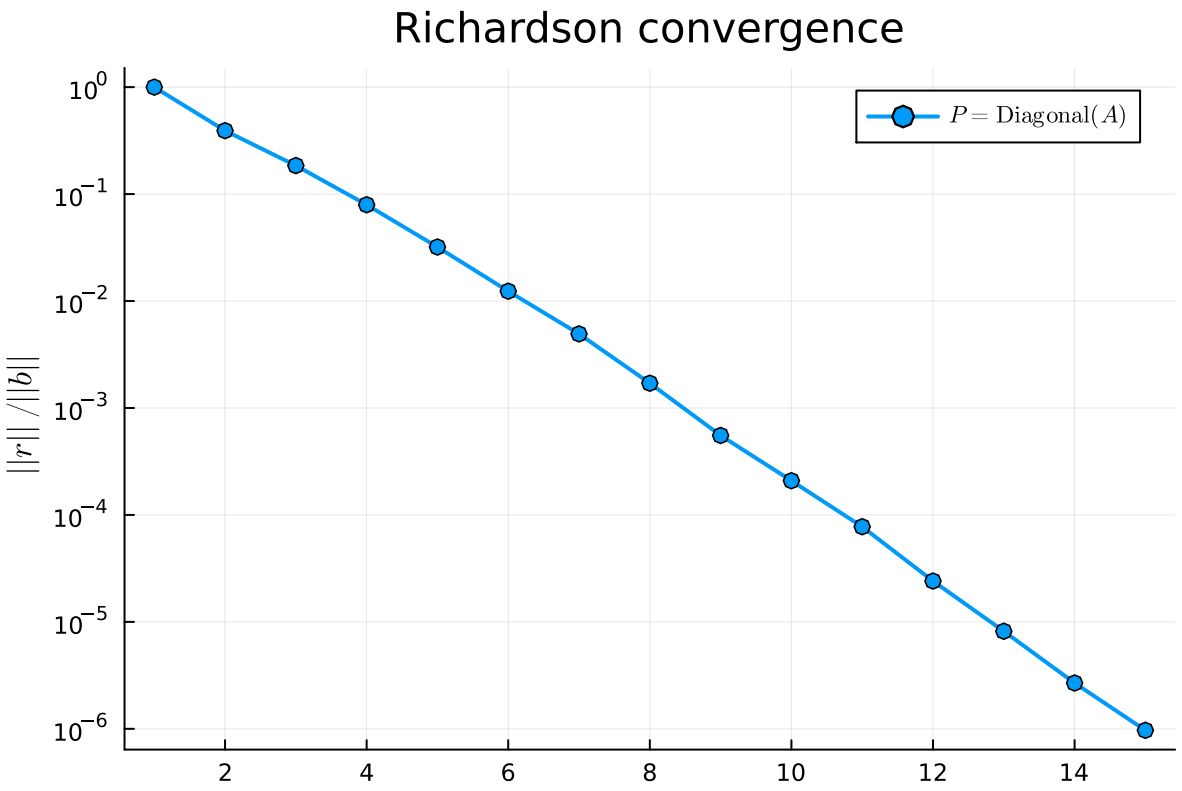
```
P = 100×100 Diagonal{Float64, Vector{Float64}}:
 13.497    ⋅        ⋅        ⋅        ⋅      ⋯   ⋅        ⋅        ⋅        ⋅
  ⋅     40.9764    ⋅        ⋅        ⋅           ⋅        ⋅        ⋅        ⋅
  ⋅       ⋅      39.8346    ⋅        ⋅           ⋅        ⋅        ⋅        ⋅
  ⋅       ⋅        ⋅      49.4443    ⋅           ⋅        ⋅        ⋅        ⋅
  ⋅       ⋅        ⋅        ⋅      22.0705       ⋅        ⋅        ⋅        ⋅
  ⋅       ⋅        ⋅        ⋅        ⋅      ⋯   ⋅        ⋅        ⋅        ⋅
  ⋅       ⋅        ⋅        ⋅        ⋅           ⋅        ⋅        ⋅        ⋅
 ⋮                                          ⋱
  ⋅       ⋅        ⋅        ⋅        ⋅           ⋅        ⋅        ⋅        ⋅
  ⋅       ⋅        ⋅        ⋅        ⋅      ⋯   ⋅        ⋅        ⋅        ⋅
  ⋅       ⋅        ⋅        ⋅        ⋅         21.8435    ⋅        ⋅        ⋅
  ⋅       ⋅        ⋅        ⋅        ⋅           ⋅      46.866     ⋅        ⋅
  ⋅       ⋅        ⋅        ⋅        ⋅           ⋅        ⋅     34.3639     ⋅
  ⋅       ⋅        ⋅        ⋅        ⋅           ⋅        ⋅        ⋅      27.5318
```

```
P = Diagonal(A)
```

Applying this preconditioner within the Richardson iterations, yields the correct result to the chosen tolerance:

```
5.0499695513836484e-8
```

```
begin
    richardson_result = richardson(A, b, P; tol=1e-6)
    maximum(abs, richardson_result.x - x_reference)
end
```




```
let
    plot(richardson_result.history;
         yaxis=:log, mark=:o, lw=2, ylabel=L"||r|| / ||b||",
         title="Richardson convergence", label=L"$P =
    \textrm{Diagonal(}A\textrm{)}$")
end
```

# Computational cost

Our main motivation for considering iterative methods was to overcome the $O(n^3)$ computational cost of Gaussian elimination / LU factorisation for solving linear systems.

Let us revist Richardson iteration (Algorithm 1) and discuss the computational cost for the case where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is a full matrix. Recall that the cost of LU factorisation scaled as $O(n^3)$ for this setting.

1. In the **first step** the most costly operation is the **computation of the matrix-vector product** $\mathbf{A}\mathbf{x}^{(k)}$. For a full matrix this costs $O(n^2)$.
2. In the **second step** we solve $\mathbf{P}\mathbf{u}^{(k)} = \mathbf{r}^{(k)}$. If $\mathbf{P}$ is a **diagonal matrix** (like we just considered), this costs $O(n)$, whereas for a **triangular matrix** (where one would perform backward or forward substitution) the cost is $O(n^2)$.
3. The computation of the new $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{u}^{(k)}$ again costs only $O(n)$.

Overall **for full matrices** the **cost of Richardson iterations** is thus only $O(n^2)$.

For **sparse matrices** $\mathbf{A}\mathbf{x}^{(k)}$ can computed in $O(n)$ computational cost. Similarly using clever preconditioners step 2 can be done in $O(n)$ time. Examples would be a sparsity-adapted forward substitution algorithm or again a diagonal preconditioner. Overall Richardson iterations thus **only cost** $O(n)$ — in stark contrast to the $O(n^3)$ cost also required for LU factorisation in this setting.

**Observation: Computational cost of iterative methods**

When solving linear systems using iterative methods, the computational cost (per iteration) scales

- for **full matrices** $\mathbf{A} \in \mathbb{R}^{n \times n}$ as $O(n^2)$
- for **sparse matrices** as $O(n)$.

A recurring theme is that the cost of the matrix-vector product $\mathbf{A}\mathbf{x}^{(k)}$ essentially determis the cost of the iterative scheme.

# ⚠ TODO ⚠

Got until here. The convergence result, which follows, is essentially what has been discussed in chapter 4, so we should use that result to save time instead of re-explaining it.

```
TODO("
Got until here.

The convergence result, which follows, is essentially what has been discussed in
chapter 4, so we should use that result to save time instead of re-explaining
it.")
```

## Convergence analysis

Having established above that Richardson iteration indeed leads to numerically the correct answer, we proceed to analyse its convergence.

Following our previous remark that Algorithm 1 can be brought into the setting of a fixed point method $\mathbf{x}^{(k+1)} = g(\mathbf{x}^{(k)})$ with

$$g(\mathbf{x}) = \mathbf{x} + \mathbf{P}^{-1}(\mathbf{b} - \mathbf{A}\mathbf{x}),$$

we study the evolution of the error $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}_*$, where $\mathbf{x}_* = g(\mathbf{x}_*)$ is the fixed point. Subtracting $\mathbf{x}_*$ from both sides of $\mathbf{x}^{(k+1)} = g(\mathbf{x}^{(k)})$ we find

$$\begin{aligned}
\mathbf{e}^{(k+1)} &= g(\mathbf{x}^{(k)}) - \mathbf{x}_* \\
&= g(\mathbf{x}^{(k)}) - g(\mathbf{x}_*) \\
&= \mathbf{x}^{(k)} + \mathbf{P}^{-1}\left(\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}\right) - \mathbf{x}_* - \mathbf{P}^{-1}(\mathbf{b} - \mathbf{A}\mathbf{x}_*) \\
&= \mathbf{e}^{(k)} + \mathbf{P}^{-1}\left(\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)} - \mathbf{b} + \mathbf{A}\mathbf{x}_*\right) \\
&= \mathbf{e}^{(k)} - \mathbf{P}^{-1}\mathbf{A}\left(\mathbf{x}^{(k)} - \mathbf{x}_*\right) \\
&= \underbrace{\left(\mathbf{I} - \mathbf{P}^{-1}\mathbf{A}\right)}_{=\mathbf{B}}\mathbf{e}^{(k)},
\end{aligned} \tag{4}$$

where $\mathbf{I}$ is the identity matrix and the matrix $\mathbf{B}$ is usually called the **iteration matrix**.

Recall the definition of the matrix norm for a matrix $M \in \mathbb{R}^{n\times n}$, namely

$$\|\mathbf{M}\| = \max_{\mathbf{0}\neq\mathbf{v}\in\mathbb{R}^n} \frac{\|\mathbf{M}\mathbf{v}\|}{\|\mathbf{v}\|}.$$

Using this and (4) we thus have

$$\begin{aligned}
\|\mathbf{e}^{(k+1)}\| &= \left\|\mathbf{B}\mathbf{e}^{(k)}\right\| \\
&\leq \|\mathbf{B}\| \left\|\mathbf{e}^{(k)}\right\| \\
&\leq \|\mathbf{B}\|^2 \left\|\mathbf{e}^{(k-1)}\right\| \\
&\vdots \\
&\leq \|\mathbf{B}\|^{k+1} \left\|\mathbf{e}^{(0)}\right\|
\end{aligned}$$

Provided $\|\mathbf{B}\| < 1$ the iteration thus converges. We summarise in a theorem:

**Theorem 1**

Given a system matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, RHS $\mathbf{b} \in \mathbb{R}^n$ and preconditioner $\mathbf{P} \in \mathbb{R}^{n\times n}$ invertible, the Richardson method with iteration matrix $\mathbf{B} = \mathbf{I} - \mathbf{P}^{-1}\mathbf{A}$ converges for *any* initial guess $\mathbf{x}^{(0)}$ if $\|\mathbf{B}\| < 1$. Moreover

$$\|\mathbf{x}_* - \mathbf{x}^{(k)}\| \leq \|\mathbf{B}\|^k \|\mathbf{x}_* - \mathbf{x}^{(0)}\|.$$

Notice that $\mathbf{B}$ is actually just the **Jacobian** of the fixed-point map $g$. In fact we can thus view the above theorem in the same light as the condition $|g'(x)| < 1$ we previously found for the convergence of the fixed-point methods for non-linear equations. However, the notable difference is that Richardson iterations converge for *any initial guess*, which for fiexd-point methods is not true in general.

The norm of iteration matrix $\|\mathbf{B}\|$ is the crucial quantity to determine not only *if* Richardson iterations converge, but also at which speed. To understand it a little better and in particular to understand the role of the preconditioner $\mathbf{P}$ we introduce the spectral radius:

**Definition: Spectral radius**

Let $\mathbf{B} \in \mathbb{R}^{n\times n}$ be a diagonalisable matrix and denote its eigenvalues as $\lambda_i(\mathbf{B})$, $i = 1, \ldots n$. The **spectral radius** of $\mathbf{B}$ is the quantity

$$\rho(\mathbf{B}) = \max_{i=1,\ldots,n} |\lambda_i(\mathbf{B})| \tag{5}$$

Note that one can show that

$$\|\mathbf{B}\|^2 = \rho(\mathbf{B}),$$

which implies

**Theorem 2**

Richardson iterations (Algorithm 1) converge if and only if $\rho(\mathbf{B}) < 1$, where $\mathbf{B} = \mathbf{I} - \mathbf{P}^{-1}\mathbf{A}$.

With this we can understand why one would even precondition at all. If we were not to perform any preconditioning, i.e. $\mathbf{P} = \mathbf{I}$, then $\rho(\mathbf{B}) = \rho\left(\mathbf{I} - \mathbf{A}\right)$ becomes

```
60.56987886053486
```

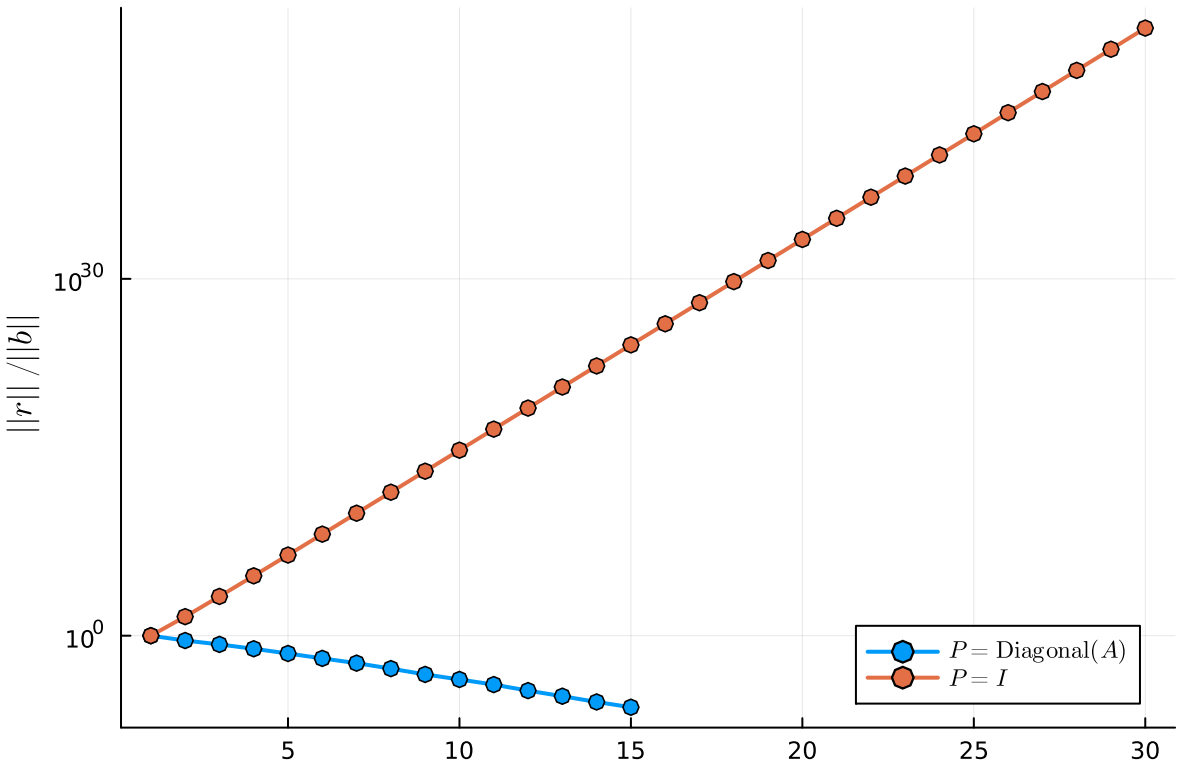
```julia
let
    B = I - A     # Iteration matrix for P = I, i.e. no preconditioner
    ρ = maximum(abs.(eigvals(B)))
end
```

whereas when using a diagonal preconditioner `P = Diagonal(A)`, i.e. $\rho(\mathbf{B}) = \rho\left(\mathbf{I} - \mathbf{P}^{-1}\mathbf{A}\right)$, then

```
0.37599909493123196
```

```julia
let
    P = Diagonal(A)
    B = I - inv(P) * A  # Iteration matrix for P = Diagonal(A)
    ρ = maximum(abs.(eigvals(B)))
end
```

the result is that only the preconditioned iterations converge:




```
let
    P = Diagonal(A)
    richardson_diagonal = richardson(A, b, P; tol=1e-6, maxiter=30)
    richardson_no_preconditioner = richardson(A, b, I; tol=1e-6, maxiter=30)

    plot(richardson_diagonal.history;
         yaxis=:log, mark=:o, lw=2, ylabel=L"||r|| / ||b||",
         label=L"$P = \textrm{Diagonal(}A\textrm{)}$", legend=:bottomright)
    plot!(richardson_no_preconditioner.history;
          label=L"$P = I$", lw=2, mark=:o)
end
```

# Choosing a good preconditioner

From the above experiments we notice:

**Observation: Preconditioners**

A good preconditioner $P$ in the Richardson iterations, satisfies the following properties:

1. It is *cheap to invert*, that is linear systems $\mathbf{P}\mathbf{u} = \mathbf{r}$ are cheap to solve.
2. The spectral radius $\rho(\mathbf{I} - \mathbf{P}^{-1}\mathbf{A})$ is as small as possible,

definitely smaller than one (to ensure convergence).

Clearly condition 2. suggests that the *ideal preconditioner* is $P = A$, such that $\mathbf{P}^{-1} = \mathbf{A}^{-1}$. In this setting $\rho(\mathbf{I} - \mathbf{P}^{-1}\mathbf{A}) = \rho(\mathbf{I} - \mathbf{I}) = 0$. The trouble of this choice is that step 2 of Algorithm 1 (Richardson iteration) now effectively requires to solve the system $\mathbf{A}\mathbf{u}^{(k)} = \mathbf{r}^{(k)}$ for $\mathbf{u}^{(k)}$, i.e. we are back to solving the original problem.

On the other hand condition 1. suggests to use $\mathbf{P}^{-1} = \mathbf{I}^{-1}$ (since the identity is cheapest to invert — nothing needs to be done). However, this does not really do anything and does thus not improve the spectral radius.

In practice we thus need to find a compromise between the two. As mentioned above standard choices for the preconditioner are:

- the diagonal of $\mathbf{A}$
- the lower triangle of $\mathbf{A}$
- another common choice is the *incomplete LU factorisation*, i.e. where one seeks a factorisation $\tilde{\mathbf{L}}\tilde{\mathbf{U}} \approx \mathbf{A}$ with a lower triangular matrix $\tilde{\mathbf{L}}$ and an upper triangular $\tilde{\mathbf{U}}$, which only *approximately* factorises $\mathbf{A}$. This gives additional freedom in designing the factorisations, in particular to avoid fill-in for sparse matrices.
- in many physics or engineering applications $\mathbf{A}$ results from a physical model. A preconditioner $\mathbf{P}$ can thus be resulting from an approximation to the employed physical model (e.g. by dropping the modelling of some physical effects).

We will not discuss the art of designing good preconditioners any further at this stage. Interested readers are referred to the excellent book Youssef Saad *Iterative methods for Sparse Linear Systems*, SIAM (2003).

# ⚠ TODO ⚠

I should be short here and make clear that this is essentially the same thing as the numerical stability discussion.

```
TODO("I should be short here and make clear that this is essentially the same thing as the numerical stability discussion.")
```

## Error control and stopping criterion

Let us return to clarifying the choice of the stopping criterion in the Richardson iterations (Algorithm 1).

As usual our goal is to control the error $\|\mathbf{x}_* - \mathbf{x}^{(k)}\|$ based on our stopping criterion, but without having access to the exact solution $x_*$. However, we know that $\mathbf{A}\mathbf{x}_* = \mathbf{b}$ , since $\mathbf{x}_*$ is just the solution to the linear system we want to solve.

Similarly $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$ directly implies

$$\mathbf{A}\mathbf{x}^{(k)} = \underbrace{\mathbf{b} - \mathbf{r}^{(k)}}_{=\widehat{\mathbf{b}}}$$

where we defined the "perturbed" RHS $\widehat{\mathbf{b}}$. Notably $\mathbf{x}^{(k)}$ is the *exact* solution of a linear system involving $\mathbf{A}$ and this perturbed RHS. Using further that

$$\|\mathbf{b}\| = \|\mathbf{A}\mathbf{x}_*\| \leq \|\mathbf{A}\| \, \|\mathbf{x}_*\| \quad \Rightarrow \quad \|\mathbf{x}_*\| \geq \frac{\|\mathbf{b}\|}{\|\mathbf{A}\|} \tag{6}$$

we develop for the relative error

$$\frac{\|\mathbf{x}_* - \mathbf{x}^{(k)}\|}{\|\mathbf{x}_*\|} \overset{(6)}{\leq} \frac{\|\mathbf{A}\| \, \left\|\mathbf{A}^{-1}\left(\mathbf{b} - \widehat{\mathbf{b}}\right)\right\|}{\|\mathbf{b}\|} \leq \|\mathbf{A}\| \, \|\mathbf{A}^{-1}\| \, \frac{\left\|\mathbf{b} - \widehat{\mathbf{b}}\right\|}{\|\mathbf{b}\|} = \|\mathbf{A}\| \, \|\mathbf{A}^{-1}\| \, \frac{\left\|\mathbf{r}^{(k)}\right\|}{\|\mathbf{b}\|}.$$

Recall the previously discussed condition number

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \, \|\mathbf{A}^{-1}\| = \sqrt{\frac{\lambda_{\max}(\mathbf{A}^T\mathbf{A})}{\lambda_{\min}(\mathbf{A}^T\mathbf{A})}}$$

as well as our stopping criterion from Algorithm 1, that is

$$\frac{\left\|\mathbf{r}^{(k)}\right\|}{\|\mathbf{b}\|} < \varepsilon,$$

With these quantities we rewrite

$$\frac{\|\mathbf{x}_* - \mathbf{x}^{(k)}\|}{\|\mathbf{x}_*\|} \leq \kappa(\mathbf{A}) \frac{\left\|\mathbf{r}^{(k)}\right\|}{\|\mathbf{b}\|} = \kappa(\mathbf{A})\, \varepsilon.$$

In other words our stopping criterion ensures that the relative error of the returned solution is no larger than $\kappa(\mathbf{A})$ times the chosen tolerance.

If the matrix is well-conditioned, i.e. $\kappa(\mathbf{A})$ is close to $1$, then the relative residual $\frac{\left\|\mathbf{r}^{(k)}\right\|}{\|\mathbf{b}\|}$ provides a good estimate of the true error and our stopping criterion is appropriate. However, if $\kappa(\mathbf{A})$ is large, even a small residual may imply a large error in the returned solution.

# Jacobi and Gauss-Seidel method

We will now briefly discuss the Jacobi and Gauss-Seidel methods, which can be seen as particular cases of Richardson iterations.

Recall equation (2)

$$\mathbf{P}\mathbf{x}^{(k+1)} = (\mathbf{P} - \mathbf{A})\mathbf{x}^{(k)} + \mathbf{b}, \qquad k = 0, 1, \ldots.$$

In the **Jacobi method** the key idea is to use the *diagonal* of the matrix $\mathbf{A}$ as the preconditioner

$$\mathbf{P} = \begin{pmatrix} A_{11} & & & \\ & A_{22} & & \\ & & \ddots & \\ & & & A_{nn} \end{pmatrix}.$$

and to explicitly insert this expression into equation (2). Rewriting we obtain in the $(k+1)$-st iteration of Jacobi's method

$$\begin{pmatrix} A_{11} & & & \\ & A_{22} & & \\ & & \ddots & \\ & & & A_{nn} \end{pmatrix} \begin{pmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{pmatrix} = - \begin{pmatrix} 0 & A_{12} & \cdots & A_{1n} \\ A_{21} & 0 & & \vdots \\ \vdots & & \ddots & \\ A_{n1} & \cdots & A_{n,n-1} & 0 \end{pmatrix} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

The diagonal structure of $\mathbf{P}$ allows the explicit computation of $\mathbf{x}^{(k+1)}$. Its $i$-th component can be obtained as

$$x_i^{(k+1)} = \frac{1}{A_{ii}} \left( b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} A_{ij} \, x_j^{(k)} \right) \tag{7}$$

for all $i = 1, \ldots, n$ and as long as $A_{ii} \neq 0$.

```
jacobi (generic function with 1 method)
```

```julia
function jacobi(A, b; x=zero(b), tol=1e-6, maxiter=100)
    history = Float64[]  # Track relative residual norm

    n  = length(x)
    xᵏ = x
    for k in 1:maxiter
        xᵏ⁺¹ = zeros(n)
        for i in 1:n
            Aᵢⱼxⱼ = 0.0
            for j in 1:n
                if i ≠ j
                    Aᵢⱼxⱼ += A[i, j] * xᵏ[j]
                end
            end  # Loop j
            xᵏ⁺¹[i] = (b[i] - Aᵢⱼxⱼ) / A[i, i]
        end  # Loop i

        relnorm_rᵏ = norm(b - A * xᵏ⁺¹) / norm(b)  # Relative residual norm
        push!(history, relnorm_rᵏ)  # Push to history
        if relnorm_rᵏ < tol         # Check convergence
            break
        end

        xᵏ = xᵏ⁺¹
    end  # Loop k
    (; x=xᵏ, history)
end
```

The **Gauss-Seidel method** employs the lower triangular part of $A$ as the preconditioner $\mathbf{P}$ (in Julia `P = LowerTriangular(A)`):

$$\mathbf{P} = \begin{pmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ \vdots & & \ddots & \\ A_{n1} & A_{n2} & \cdots & A_{nn} \end{pmatrix}.$$

By inserting the lower-triangular $\mathbf{P}$ into (2) we obtain in the $(k+1)$-st iteration of Gauss-Seidel:

$$\begin{pmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ \vdots & & \ddots & \\ A_{n1} & A_{n2} & \cdots & A_{nn} \end{pmatrix} \begin{pmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{pmatrix} = - \begin{pmatrix} 0 & A_{12} & \cdots & A_{1n} \\ & 0 & \ddots & \vdots \\ & & \ddots & A_{n-1,n} \\ & & & 0 \end{pmatrix} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}.$$

Using forward substitution to solve this linear system leads to the following form for the $i$-th component of the vector $\mathbf{x}^{(k+1)}$.

$$x_i^{(k+1)} = \frac{1}{A_{ii}} \left( b_i - \sum_{j=1}^{i-1} A_{ij}\, x_j^{(k+1)} - \sum_{j=i+1}^{n} A_{ij}\, x_j^{(k)} \right) \tag{8}$$

for all $i = 1, \ldots, n$ and as long as $A_{ii} \neq 0$.

Notice that Gauss-Seidel is very similar to Jacobi's method, just with the small difference that for computing the new component $x_i^{(k+1)}$ we use the components $x_j^{(k+1)}$ for $j < i$, which have already been updated to their new values.

```
gauss_seidel (generic function with 1 method)
```

```julia
function gauss_seidel(A, b; x=zero(b), tol=1e-6, maxiter=100)
    history = Float64[]  # Track relative residual norm

    n  = length(x)
    xᵏ = x
    for k in 1:maxiter
        xᵏ⁺¹ = zeros(n)
        for i in 1:n
            Aᵢⱼxⱼ = 0.0
            for j in 1:i-1
                Aᵢⱼxⱼ += A[i, j] * xᵏ⁺¹[j]
            end  # Loop j
            for j in i+1:n
                Aᵢⱼxⱼ += A[i, j] * xᵏ[j]
            end  # Loop j
            xᵏ⁺¹[i] = (b[i] - Aᵢⱼxⱼ) / A[i, i]
        end  # Loop i

        relnorm_rᵏ = norm(b - A * xᵏ⁺¹) / norm(b)  # Relative residual norm
        push!(history, relnorm_rᵏ)  # Push to history
        if relnorm_rᵏ < tol         # Check convergence
            break
        end

        xᵏ = xᵏ⁺¹
    end  # Loop k
    (; x=xᵏ, history)
end
```

These two cases just provide two examples of the many flavours of Richardson iterations, which are used in practice. A good overview provides chapter 4 of Youssef Saad *Iterative methods for Sparse Linear Systems*, SIAM (2003).

We return to our example problem $\mathbf{Ax} = \mathbf{b}$ with

```
100×100 Matrix{Float64}:
 13.497     -0.0373554   0.372059    0.455755   …  -0.993431    1.72989   -0.500089
  0.339416  40.9764     -0.49252    -0.632024      0.0242995   1.32131    0.53068
  0.883111  -0.238479   39.8346      0.082532     -0.27617     0.522972  -1.09944
 -0.860942   1.01789    -0.405506   49.4443        1.36896    -1.25262    0.131916
 -0.910278   2.46261     0.0137423  -0.0335643    -0.0064238  -0.652895   0.485895
  1.99898   -1.65667    -0.0638512  -3.35575    …  0.390615    0.238168   0.693313
  0.930066   0.801534    1.35834     0.523876     -0.436435   -0.866964   0.239833
  ⋮                                              ⋱
  0.639166  -0.136747   -0.163707   -0.109682     -0.190203   -0.528371   0.946597
  0.270023  -0.186965   -0.888258    0.0374101  …  -1.30121     1.30031   -0.516153
 -1.24605   -0.348588   -1.03105    -0.276275      1.21323     0.394299  -1.35424
 -1.13901   -1.02876    -1.02613     0.158151     46.866       0.480092  -0.478084
 -0.553396  -0.264033    1.19852    -0.626974      0.739913   34.3639     1.75868
  1.05441    0.760557   -0.669326    0.381245     -1.04612    -1.48061   27.5318
```

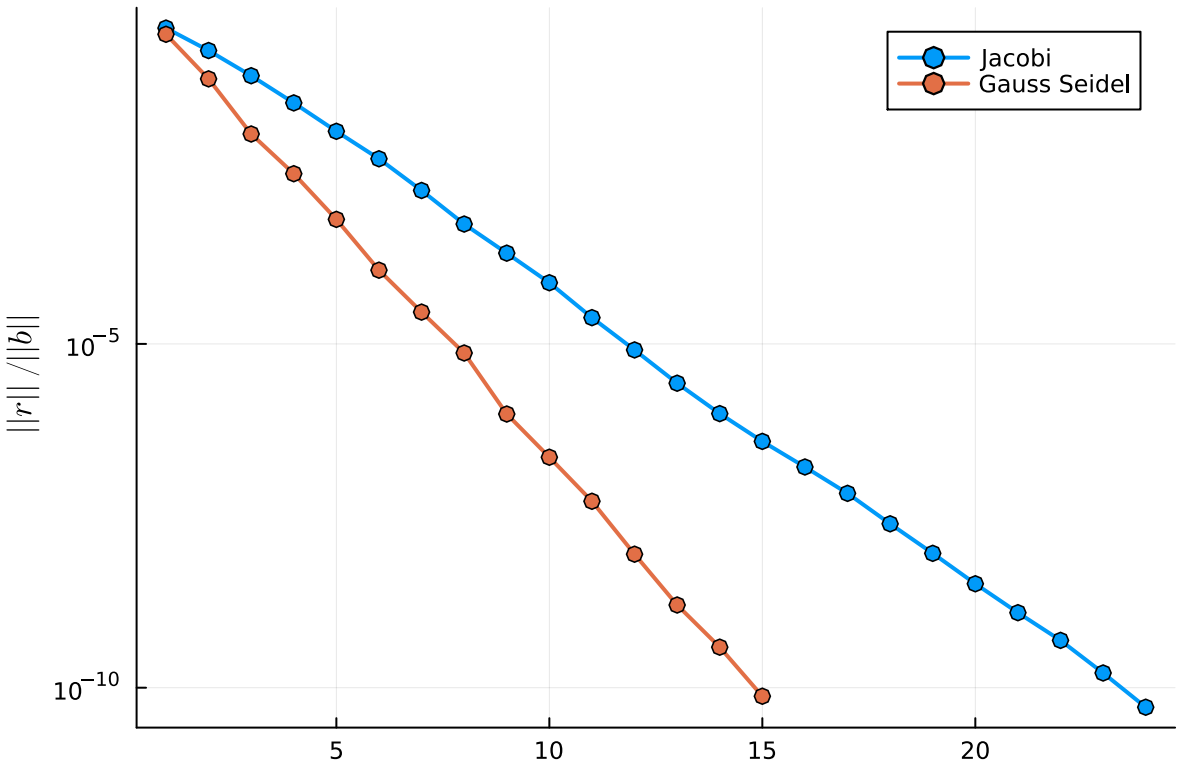
```
A
```

and

```
[0.0742384, 0.0760846, 0.989216, 0.305673, 0.157284, 0.350807, 0.879947, 0.50237, 0.819
```

```
b
```

On this problem the convergence is as follows:




```
let
    result_jacobi       = jacobi(A, b; tol=1e-10, maxiter=30)
    result_gauss_seidel = gauss_seidel(A, b; tol=1e-10, maxiter=30)


    P = Diagonal(A)
    richardson_diagonal = richardson(A, b, P; tol=1e-6, maxiter=30)
    richardson_no_preconditioner = richardson(A, b, I; tol=1e-6, maxiter=30)

    plot(result_jacobi.history;
         yaxis=:log, mark=:o, lw=2, ylabel=L"||r|| / ||b||",
         label="Jacobi", legend=:topright)
    plot!(result_gauss_seidel.history;
          label="Gauss Seidel", lw=2, mark=:o)
end
```

# Linear systems involving symmetric positive-definite matrices

In many applications in engineering and the sciences the arising system matrices $\mathbf{A}$ have additional properties, which can be exploited to obtain better-suited algorithms. An important case are the symmetric positive definite matrices.

This part of the notes only provides a condensed introduction. A more detailed, but very accessible introduction to the steepest descent and conjugate gradient methods discussed in this section provides Jonathan Shewchuk *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain* (1994).

Let us first recall the definition of symmetric positive definite matrices.

**Definition: Positive definite**

A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is called **positive definite** if

$$\forall \mathbf{0} \neq \mathbf{v} \in \mathbb{R}^n \qquad \mathbf{v}^T \mathbf{A} \mathbf{v} > 0$$

For **symmetric matrices** we additionally have that $\mathbf{A}^T = \mathbf{A}$. Symmetric positive definite matrices (often abbreviated as s.p.d. matrices) arise naturally in physical systems when looking for configurations minimising their energy. A result underlining this construction is

**Theorem 2**

An matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is s.p.d. if and only if all its eigenvalues are real and positive.

To every linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$ involving an s.p.d. system matrix $\mathbf{A}$ we can associate an **energy function** $\phi$

$$\phi : \mathbb{R}^n \to \mathbb{R} : \phi(\mathbf{v}) = \frac{1}{2}\mathbf{v}^T \mathbf{A} \mathbf{v} - \mathbf{v}^T \mathbf{b}$$

Using basic vector calculus one shows that

$$\begin{aligned} \textbf{Gradient} \qquad & \nabla \phi(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} = -\mathbf{r} && \textit{(negative residual)} \\ \textbf{Hessian} \qquad & H_\phi(\mathbf{x}) = \mathbf{A} && \textit{(system matrix)} \end{aligned} \tag{9}$$

Setting the gradient to zero, we obtain the stationary points. The corresponding equation

$$\mathbf{0} = \nabla \phi(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$$

only has a single solution, provided that $\mathbf{A}$ is non-singular. Since the Hessian $H_\phi(\mathbf{x}) = \mathbf{A}$ is positive definite, this stationary point is a minimum. As a result we obtain

**Proposition 3**

Given an s.p.d. matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the solution of the system $\mathbf{A}\mathbf{x}_* = \mathbf{b}$ is the unique minimum of the function $\phi$, i.e.

$$\mathbf{x}_* = \mathbf{argmin}_{\mathbf{v} \in \mathbb{R}^n} \phi(\mathbf{v}). \tag{10}$$

## Steepest descent method

# ⚠ TODO ⚠

More details

```julia
TODO("More details")
```

A Julia implementation of steepest descent is given by

```
steepest_descent_simple (generic function with 1 method)
```

```julia
function steepest_descent_simple(A, b; x=zero(b), tol=1e-6, maxiter=100)
    history = Float64[]  # Track relative residual norm

    r = b - A * x
    for k in 1:maxiter
        relnorm = norm(r) / norm(b)
        push!(history, relnorm)
        if relnorm < tol
            break
        end

        Ar = A * r
        α = dot(r, r) / dot(r, Ar)
        x = x + α * r
        r = r - α * Ar
    end
    (; x, history)
end
```

```
100×100 Matrix{Float64}:
  8.06563    -0.770906    0.0565932  …   0.451446    0.370626   -0.142734
 -0.770906    8.31006    -0.10638        1.07475    -0.363524   -0.367626
  0.0565932  -0.10638     9.32111        0.626578    0.0213407   0.0796574
 -0.0728217  -0.0176772  -0.669367       0.677923   -0.191499    0.22379
 -0.0641255  -0.208026   -0.0731268     -0.181866    0.107675    0.658375
  0.0253099   1.17779     0.395487   …  -0.0610738  -0.631199   -0.227752
 -0.715817    0.319745    0.124714       1.17606    -0.695958   -0.0835588
  ⋮                                   ⋱
 -0.52945     0.868638    0.510023       0.337998   -0.0955734   0.468059
  0.0887523  -0.536024    0.0490932  …  -0.321766   -0.0996351   0.179948
 -0.594778    0.36869     0.0684089      0.487577   -0.12406    -0.0332497
  0.451446    1.07475     0.626578       8.70709    -0.326609    0.671578
  0.370626   -0.363524    0.0213407     -0.326609    9.02844    -0.528092
 -0.142734   -0.367626    0.0796574      0.671578   -0.528092    8.01639
```

```julia
begin
    # Generate a random s.p.d. matrix
    U, Σ, V = svd(randn(100, 100))
    Aₛ = U * Diagonal(abs.(Σ)) * U'
end
```

```julia
bₛ = rand(100);
```

The system matrix has a condition number larger well larger than $1$:

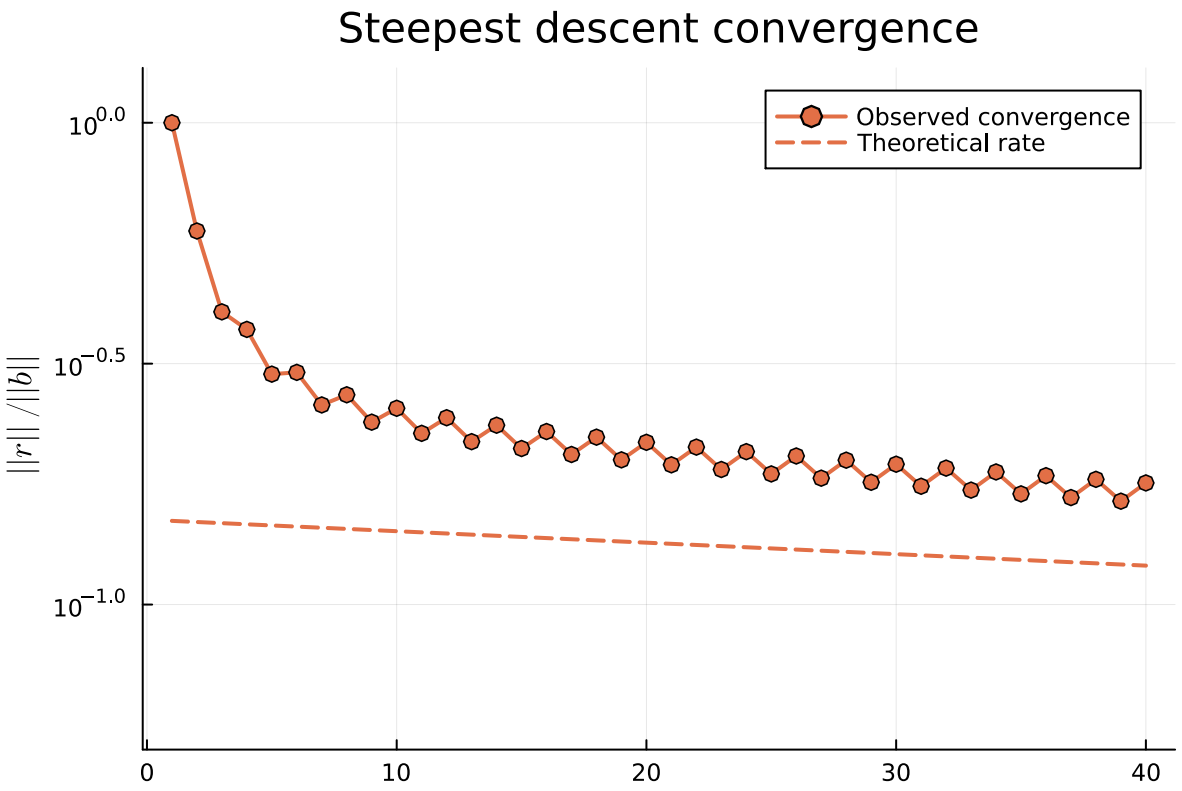
```
364.62565489245696
```

```julia
cond(Aₛ)
```

Still, as the theory predicts, the method converges. However, convergence gets slow after a while and approaches the theoretical rate

```
rate_steep_desc = 0.994529924327689
```

```julia
rate_steep_desc = (cond(Aₛ) - 1) / (cond(Aₛ) + 1)
```




```
let
    descent = steepest_descent_simple(Aₛ, bₛ; tol=1e-6, maxiter=40)

    plot(descent.history;
        yaxis=:log, mark=:o, lw=2, ylabel=L"||r|| / ||b||", ylims=(5e-2, 1.3),
        title="Steepest descent convergence", label="Observed convergence", c=2)

    plot!(1:40, x -> 0.15rate_steep_desc^x, lw=2, ls=:dash, label="Theoretical
    rate", c=2)
end
```

# Preconditioned steepest descent

# ⚠ TODO ⚠

More details

```
TODO("More details")
```

An implementation is

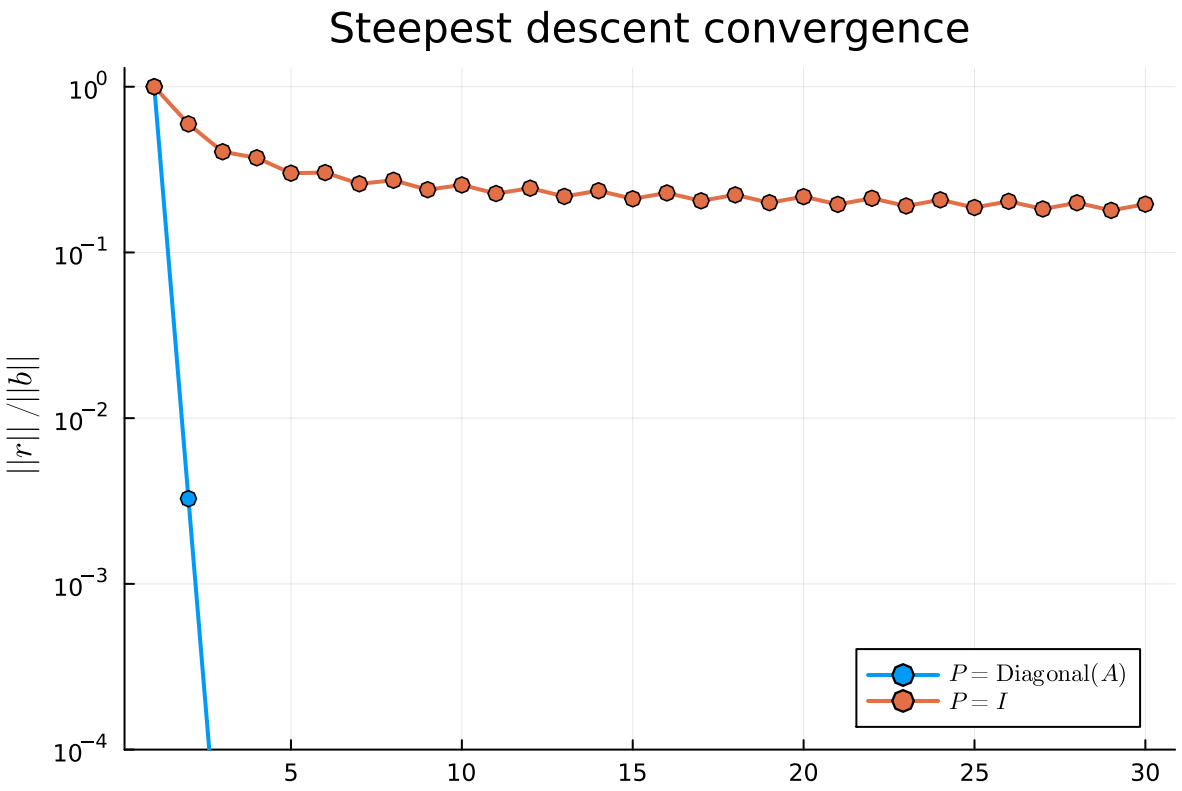
```
steepest_descent (generic function with 1 method)
```

```
function steepest_descent(A, b, P; x=zero(b), tol=1e-6, maxiter=100)
    history = Float64[]

    r = b - A * x
    for k in 1:maxiter
        relnorm = norm(r) / norm(b)
        push!(history, relnorm)
        if relnorm < tol
            break
        end

        z = P \ r  # Solve Pz = r  for z, the main addition compared to
                   # steepest_descent_simple in this function
        Az = A * z
        α = dot(r, r) / dot(r, Az)
        x = x + α * r
        r = r - α * r
    end
    (; x, history)
end
```

We consider the convergence of both the preconditioned and the non-preconditioned steepest descent method on the model problem $\mathbf{A}_s\mathbf{x} = \mathbf{b}_s$.




```
let
    P = Diagonal(Aₛ)
    descent_diagonal = steepest_descent(Aₛ, bₛ, P; tol=1e-6, maxiter=30)
    descent_no_preconditioner = steepest_descent_simple(Aₛ, bₛ; tol=1e-6, maxiter=30)

    plot(descent_diagonal.history;
         yaxis=:log, mark=:o, lw=2, ylabel=L"||r|| / ||b||",
         label=L"$P = \textrm{Diagonal(}A\textrm{)}$", legend=:bottomright,
         title="Steepest descent convergence", ylims=(1e-4, 1.3))
    plot!(descent_no_preconditioner.history;
          label=L"$P = I$", lw=2, mark=:o)
end
```

# Conjugate gradient method

We only show the conjugate gradient method *without preconditioning*.

```
md"""
We only show the conjugate gradient method *without preconditioning*.
"""
```

# ⚠ TODO ⚠

Show a classic case where steepest descent fails and CG works, e.g. a Rosenbrock or a cartesian product of a steep and a shallow parabola.

```
TODO("Show a classic case where steepest descent fails and CG works, e.g. a Rosenbrock or a cartesian product of a steep and a shallow parabola.")
```

# ⚠ TODO ⚠

More details

```
TODO("More details")
```

For this example the theoretical rate is:

```
rate_cg = 0.9004736142316335
```

```
rate_cg = (sqrt(cond(Aₛ)) - 1) / (sqrt(cond(Aₛ)) + 1)
```

A (non-optimised) CG implementation is:

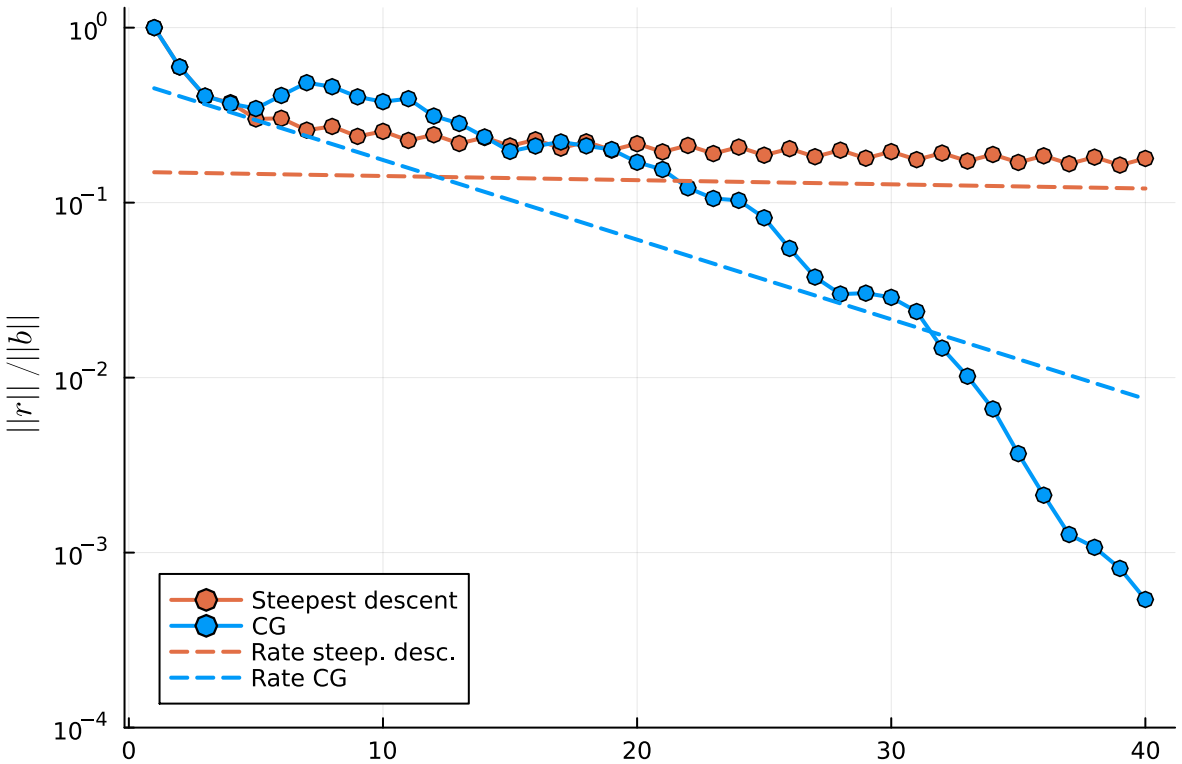
```
conjugate_gradient_simple (generic function with 1 method)
```

```
function conjugate_gradient_simple(A, b; x=zero(b), tol=1e-6, maxiter=100)
    history = Float64[]  # Track relative residual norm

    r = b - A * x
    p = r
    for k in 1:maxiter
        relnorm = norm(r) / norm(b)
        push!(history, relnorm)
        if relnorm < tol
            break
        end

        # Descent along conjugate direction p (instead of along r)
        Ap = A * p
        α  = dot(r, r) / dot(p, Ap)
        x  = x + α * p
        r_new = r - α * Ap

        # Update conjugate direction p
        β = dot(r_new, r_new) / dot(r, r)
        p = r_new + β * p
        r = r_new
    end
    (; x, history)
end
```




```julia
let
    P = Diagonal(A)
    descent_no_preconditioner = steepest_descent_simple(Aₛ, bₛ; tol=1e-6, maxiter=40)
    cg_no_preconditioner = conjugate_gradient_simple(Aₛ, bₛ; tol=1e-6, maxiter=40)

    plot(descent_no_preconditioner.history;
         yaxis=:log, mark=:o, lw=2, ylabel=L"||r|| / ||b||",
         label="Steepest descent", legend=:bottomleft, c=2,  ylims=(1e-4, 1.3))
    plot!(cg_no_preconditioner.history; label="CG", lw=2, mark=:o, c=1)


    plot!(1:40, x -> 0.15rate_steep_desc^x, lw=2, ls=:dash, label="Rate steep. desc.", c=2)
    plot!(1:40, x -> 0.5rate_cg^x, lw=2, ls=:dash, label="Rate CG", c=1)
end
```

**Numerical analysis**


1. Introduction
2. The Julia programming language
3. Revision and preliminaries
4. Root finding and fixed-point problems
5. Interpolation

6. Direct methods for linear systems
7. Iterative methods for linear systems
8. Eigenvalue problems
9. Numerical integration
10. Numerical differentiation
11. Initial value problems